Faculté des Sciences et Techniques de Limoges

M1 - GPGPU
TP1 - Premiers pas en CUDA

Maxime MARIA, Vincent LARROQUE

2024-2025

Ce sujet a pour but de vous initier à la programmation parallèle sur processeurs graphiques (GPU) en utilisant CUDA. CUDA (Compute Unified Device Architecture) est une technologie propriétaire proposée par NVidia qui permet d'exploiter la puissance de calcul massivement parallèle intrinsèque aux GPU.

Évidemment, pour pouvoir correctement appréhender les notions abordées, vous devez vous référer au cours (`http://p-fb.net/master1/gpgpu/`) mais également à la documentation officielle (`https://docs.nvidia.com/cuda/index.html`).

Pour compiler un programme CUDA, vous pouvez utilisez la commande suivante :

```
$ nvcc -o nom_prog main.cu [dependances]
```

Les noms des fichiers .cpp ou .cu contenant les dépendances sont à ajouter, séparés par un espace, après le nom du fichier contenant la fonction `main`.

1 Vérification du matériel

1. Téléchargez le dossier `TP1-Ex1-Verification-du-materiel`.
2. Compilez et exécutez le programme donné afin d'afficher les différentes propriétés de la carte graphique utilisée. Vous devriez voir apparaître les informations relatives au(x) GPU de votre ordinateur.

2 Addition de vecteurs : $\vec{a} + \vec{b} = \vec{c}$

Dans cet exercice, vous allez additionner deux vecteurs d'entiers en parallèle.

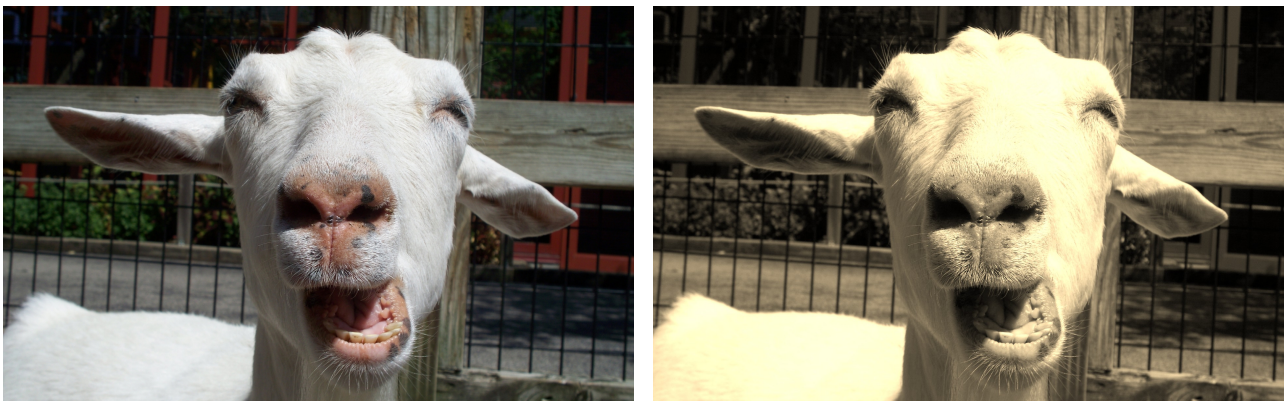
1. Téléchargez le dossier `TP1-Ex2-Addition-de-vecteurs`. Dans ce dossier, vous ne devrez modifier **QUE** les fichiers *student.hpp* et *student.cu*. Les fichiers *reference.hpp* et *reference.cpp* contiennent le calcul de l'addition en séquentiel sur CPU. Le fichier *main.cu* contient le programme principal qui appelle les fonctions de calcul, mesure les temps d'éxecution et compare les résultats CPU/GPU.
2. Complétez les fichiers *student.hpp* et *student.cu* afin de :
   - allouer de la mémoire sur le GPU : `allocateArraysCUDA` ;
   - transférer les données du CPU vers le GPU : `copyFromHostToDevice` ;
   - configurer la répartition des *threads* et lancer le *kernel* : `launchKernel` ;
   - récupérer les données sur le GPU : `copyFromDeviceToHost` ;
   - libérer la mémoire : `freeArraysCUDA`.

Pour chaque appel à une fonction CUDA, utilisez la macro `HANDLE_ERROR` disponible dans le fichier `commonCUDA.hpp`. Celle-ci vous permet de récupérer les erreurs CUDA pendant l'exécution de vos programmes.

3. Écrivez le *kernel* `sumArraysCUDA` qui réalise l'addition.
4. Votre *kernel* doit fonctionner quelque soient les nombres de *blocks* et de *threads* choisis. Si ce n'est pas le cas, modifiez votre *kernel* en conséquence.
5. Comparez les temps d'exécution entre les différentes versions (CPU/GPU) en faisant varier :
   - la taille des vecteurs : donnez la taille en argument du programme avec l'option `-n` (attention à ne pas dépasser la capacité mémoire de votre GPU !) ;
   - les nombres de *blocks* et de *threads*.

3 Filtre d'images sépia

Dans cet exercice, vous devez implémentez un algorithme qui, à partir d'une image au format `.ppm`, applique un filtre afin de produire une nouvelle ayant un aspect sépia (*cf.* Figure 1).



(a) (b)

FIGURE 1 – Filtre d'image sépia : (a) Image originale ; (b) Image sépia.

L'image source est passée en argument du programme à l'aide de l'option `-f image_path.ppm`.

L'algorithme de filtre est simple et consiste à modifier la couleur de chaque pixel tel que :

```
outRed = min(255, (inRed * .393f + inGreen * .769f + inBlue * .189f));
outGreen = min(255, (inRed * .349f + inGreen * .686f + inBlue * .168f));
outBlue = min(255, (inRed * .272f + inGreen * .534f + inBlue * .131f));
```

1. Téléchargez le dossier `TP1-Ex3-Filtre-d-images-sepia`. Une nouvelle fois, vous devrez modifier **QUE** les fichiers *student.hpp* et *student.cu*.
2. Complétez la fonction `sepiaGPU` (allouez/transférez/libérez la mémoire, *etc.*). Cette fois-ci, vous devez **OBLIGATOIREMENT** configurer la répartition des *threads* sur une grille 2D (nous aurions pu le faire en 1D oui, mais non !).
3. Implémentez le *kernel* `kernelComputeSepia` et vérifiez son bon fonctionnement.
4. Votre *kernel* doit fonctionner quelque soient les nombres de *blocks* et de *threads* choisis. Si ce n'est pas le cas, modifiez votre *kernel* en conséquence.
5. Comparez et analysez les temps d'exécution mesurés pour des images de tailles différentes et avec des nombres de *blocks* et de *threads* différents. Des images sont fournies dans l'archive du TP. Cependant, n'hésitez pas à tester vos algorithmes avec d'autres images au format `.ppm`.